

Logistic Regression

June 24, 2025

Introduction

Logistic regression is another technique borrowed by machine learning from the field of statistics. It is the go-to method for binary classification problems (problems with two class values).

In this note, you will discover the logistic regression algorithm for machine learning. After reading this note, you will know:

- The representation used for a logistic regression model.
- Techniques used to learn the coefficients of a logistic regression model from data.
- How to actually make predictions using a learned logistic regression model.

1 Logistic Function

Logistic regression is named for the function used at the core of the method, the logistic function. The logistic function, also called the sigmoid function, was developed by statisticians to describe properties of population growth in ecology, rising quickly and maxing out at the carrying capacity of the environment. It is an S-shaped curve that can take any real-valued number and map it into a value between 0 and 1, but never exactly at those limits.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where e is the base of the natural logarithms (Euler's number) and z is the actual numerical value that you want to transform.

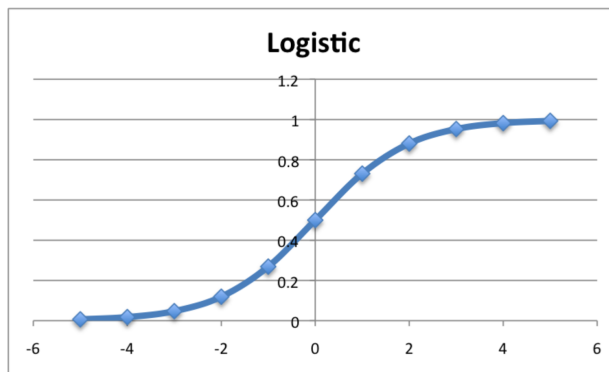


Figure 1: Logistic Function.

2 Representation used for Logistic Regression

Logistic regression uses an equation as the representation, very much like linear regression. Input values (x) are combined linearly using weights or coefficient values to predict an output value (y). A key difference from linear regression is that the output value being modeled is a binary value (0 or 1) rather than a continuous numeric value.

Below is an example logistic regression equation:

$$y = \frac{e^{\theta_0 + \theta_1 x}}{1 + e^{\theta_0 + \theta_1 x}}$$

Where y is the predicted output, θ_0 is the bias or intercept term, and θ_1 is the coefficient for the single input value x . Each column in your input data has an associated θ coefficient (a constant real value) that must be learned from your training data. The actual representation of the model that you would store in memory or in a file consists of these coefficients in the equation.

3 Logistic Regression Predicts Probabilities

Logistic regression models the probability of the default class (e.g., the first class). For example, if we are modeling a person's sex as male or female from their height, then the first class could be *male*, and the logistic regression model could be written as the probability of being male given a person's height, or more formally:

$$Pr(\text{sex} = \text{male} \mid \text{height})$$

Written another way, we are modeling the probability that an input x belongs to the default class ($y = 1$). We can write this formally as:

$$P(x) = \text{Class probability of } x = Pr(y = 1 \mid x)$$

We are predicting probabilities? I thought logistic regression was a classification algorithm? Note that the probability prediction must be transformed into a binary value (0 or 1) in order to actually make a crisp prediction. More on this later when we discuss making predictions.

Logistic regression is a linear method, but the predictions are transformed using the logistic function. The impact of this is that we can no longer interpret the predictions as a simple linear combination of the inputs as we can with linear regression. For example, continuing from the above, the model can be stated as:

$$\text{Class probability of } x = \frac{e^{\theta_0 + \theta_1 x}}{1 + e^{\theta_0 + \theta_1 x}}$$

We would not dive too deep into the math, but we can rearrange the above equation as follows (remember we can remove the exponentiation by applying the natural logarithm $\ln()$ to both sides):

$$\ln \left(\frac{p(x)}{1 - p(x)} \right) = \theta_0 + \theta_1 x$$

This is useful because we can now see that the calculation on the right-hand side is linear again (just like in linear regression), and the input on the left-hand side is the natural logarithm of the probability of the default class.

The left-hand side is referred to as the *logit*. It is possible to use other functions for this transformation (which is outside the scope of this discussion), but in general, this transformation that relates the linear regression equation to the probabilities is called the *link function*, e.g., the *logit link function*.

4 Learning the Logistic Regression Model

The coefficients of the logistic regression algorithm must be estimated from your training data. This is done using *maximum-likelihood estimation*. Maximum-likelihood estimation is a common learning algorithm used by a variety of machine learning techniques, although it does make assumptions about the distribution of your data (more on this when we discuss preparing your data).

The best coefficients would result in a model that predicts a value very close to 1 (e.g., *male*) for the default class and a value very close to 0 (e.g., *female*) for the other class. The intuition behind maximum-likelihood for logistic regression is that a search procedure seeks values for the coefficients that minimize the error in the probabilities predicted by the model compared to those observed in the data (e.g., predicting a probability of 1 when the true class is the primary class).

We are not going to delve into the mathematics of maximum-likelihood estimation here. It is enough to say that a minimization algorithm is used to optimize the best values for the coefficients given your training data. This is often implemented in practice using efficient numerical optimization algorithms, such as the *Quasi-Newton method*, like an LBFGS algorithm.

When learning logistic regression from scratch, one can implement the optimization using a much simpler approach: *gradient descent*.

5 Making Predictions with Logistic Regression

Making predictions with a logistic regression model is as simple as plugging numbers into the logistic regression equation and calculating a result. Let's make this concrete with a specific example.

Suppose we have a model that predicts whether a person is *male* or *female* based on their height (purely hypothetical). Given a height of 150 cm, is the person male or female?

We assume the learned coefficients are $\theta_0 = -100$ and $\theta_1 = 0.6$. Using the logistic regression equation, we can calculate the probability of the person being male given their height of 150 cm, or more formally, $Pr(\text{male} \mid \text{height} = 150)$.

We use $\text{EXP}()$ to denote the exponential function e^x , as commonly available in spreadsheets:

$$y = \frac{e^{\theta_0 + \theta_1 x}}{1 + e^{\theta_0 + \theta_1 x}}$$
$$y = \frac{\text{EXP}(-100 + 0.6 \times 150)}{1 + \text{EXP}(-100 + 0.6 \times 150)}$$
$$y = 0.0000453978687$$

This results in a probability near zero that the person is male.

In practice, we can use the probabilities directly. However, because this is a classification task and we want a crisp answer, we typically convert the probability into a binary class value. For example:

$$\text{prediction} = \begin{cases} 0 & \text{if } Pr(\text{male}) < 0.5 \\ 1 & \text{if } Pr(\text{male}) \geq 0.5 \end{cases}$$

Now that we know how to make predictions using logistic regression, let us look at how we can prepare our data to get the most out of this technique.

6 Prepare Data for Logistic Regression

The assumptions made by logistic regression about the distribution and relationships in your data are much the same as the assumptions made in linear regression. Much study has gone into defining these assumptions using precise probabilistic and statistical language.

My advice is to use these as guidelines or rules of thumb and experiment with different data preparation schemes. Ultimately, in predictive modeling and machine learning projects, your focus is on making accurate predictions rather than interpreting the results. As such, you can break some assumptions as long as the model is robust and performs well.

- **Binary Output Variable:** This might be obvious as we have already mentioned it, but logistic regression is intended for binary (two-class) classification problems. It will predict the probability of an instance belonging to the default class, which can then be snapped into a 0 or 1 classification.

- **Remove Noise:** Logistic regression assumes no error in the output variable (y), so consider removing outliers and possibly misclassified instances from your training data.
- **Gaussian Distribution:** Logistic regression is a linear algorithm (with a nonlinear transform on the output). It assumes a linear relationship between the input variables and the output. Transformations of your input variables that better expose this linear relationship can result in a more accurate model. For example, you can use log, root, Box-Cox, and other univariate transforms to better expose this relationship.
- **Remove Correlated Inputs:** Like linear regression, the model can overfit if you have multiple highly correlated inputs. Consider calculating the pairwise correlations between all inputs and removing those that are highly correlated.
- **Fail to Converge:** It is possible for the maximum likelihood estimation process used to learn the coefficients to fail to converge. This can occur if there are many highly correlated inputs in your data or if the data is very sparse (e.g., lots of zeros in your input data).